Namespace PhotoCatalog.Application.Caching

Classes

CacheKeysFactory

Централизованная фабрика строковых идентификаторов для кэширования. Разделяет генерацию Key и Tag.

Полное Техническое Задание на разработку библиотеки каталогизации фотографий

1. Назначение документа и описание проекта

Назначение: Документ описывает архитектуру, функциональные требования и структуру компонентов для системы каталогизации фотографий.

Суть проекта: Разработка ядра (библиотеки) для управления коллекцией фотографий с использованием концепции "виртуальных альбомов". Физические файлы хранятся в единой директории, а иерархия папок, альбомов и тегов формируется исключительно на логическом уровне в базе данных.

2. Архитектурные требования и стек технологий

Система проектируется с соблюдением принципов **Чистой Архитектуры**, **SOLID** и **Богатой Доменной Модели**.

- **Основная платформа:** .NET 10
- **Язык программирования:** C# 13
- **База данных:** SQLite
- **Доступ к данным: Dapper**. Используется как легковесный ORM для выполнения SQL-запросов и маппинга результатов в доменные объекты, обеспечивая баланс между производительностью, контролем над SQL и удобством разработки.
- **Логирование: Serilog**. Используется для структурированного логирования бизнес-потоков, отслеживания действий пользователя и фиксации системных сбоев.
- **Тестирование:** xUnit (Unit и интеграционные тесты), ArchUnitNET (архитектурные тесты).
- **Инверсия зависимостей (DIP):** Внутренние слои (`Domain`) абсолютно независимы от внешних (`Infrastructure`, `Application`, сторонние библиотеки). Взаимодействие происходит строго через контракты (интерфейсы).

3. Функциональные требования

3.1. Управление физическими файлами

- **Импорт:** Система принимает путь к файлу, вычисляет его хэш, извлекает метаданные (размер, ширина, высота) и копирует/перемещает его в единую целевую директорию хранилища.

- **Синхронизация:** Система проверяет наличие физического файла по сохраненному пути и помечает записи в БД как "осиротевшие", если файл был удален средствами ОС.
- **Физическое удаление:** При явном удалении фотографии из системы, виртуальные связи в БД уничтожаются каскадно в рамках транзакции. Физический файл удаляется с диска строго **после** успешного коммита транзакции в базе данных.

3.2. Управление виртуальной иерархией

- **Виртуальные папки:** Создание, переименование, удаление и поддержка вложенности (папка внутри папки). На диске эти папки не создаются. Защита от создания циклических зависимостей (помещение папки внутрь своего потомка) контролируется на слое `Application`.
- **Виртуальные альбомы:** Создание альбомов внутри папок или в корне виртуального дерева.
- **Управление контентом:** Добавление фотографии в альбом (создание логической связи). Одна фотография может одновременно находиться в неограниченном количестве альбомов. Удаление фото из альбома удаляет только связь, физический файл остается нетронутым.

3.3. Тегирование и поиск

- **Тегирование:** Создание строковых тегов и привязка их к фотографиям.
- **Поиск и фильтрация:** Получение списка фотографий по пересечению условий:
  - Нахождение в определенном виртуальном альбоме или папке.
  - Наличие всех или хотя бы одного из указанных тегов.

4. Структура проекта (Слои)

Директории (папки) в решении должны именоваться строго в **PascalCase**. Проект разделен на три основных слоя:

4.1. PhotoCatalog.Domain (Ядро)

Не имеет никаких внешних зависимостей. Содержит чистую бизнес-логику. **Свободен от любых библиотек логирования.**

- **Entities:** `Photo`, `Album`, `Folder`, `Tag`. Инкапсулируют состояние (используются `private` или `init`-only сеттеры). Логика изменения состояния находится внутри самих классов и возвращает объекты `Result` / `ResultVoid` вместо выброса исключений при нарушении бизнес-правил.

- **Value Objects:** Неизменяемые объекты без идентичности (например, `Dimensions` для габаритов фото).
- **Errors Registry:** Реестр доменных ошибок для контроля бизнес-правил и инвариантов (например, `DomainErrors.Tag.TooLong`).
- **Interfaces:** `IPhotoRepository`, `IFolderRepository`, `IFileStorage`, `IUnitOfWork`. Контракты репозиториев возвращают `Result<T>` или `ResultVoid` для обеспечения консистентности ROP.

4.2. PhotoCatalog.Application (Сценарии использования)

Зависит только от `Domain`. Является основным местом для логирования бизнес-потоков.

- **Use Cases (Services):** Обработчики команд (например, `ImportPhotoUseCase`). Оркестрируют логику, **логируют** начало важных операций, их успешное завершение и фиксируют провальные `Result` (с уровнями Warning/Information) без остановки приложения.
- **DTO:** Простые структуры данных для возврата результатов на уровень представления.

4.3. PhotoCatalog.Infrastructure (Реализация)

Зависит от `Domain` и `Application`. Отвечает за логирование критических системных сбоев.

- **Data Access:** Реализация репозиториев для SQLite с использованием **Dapper**. Реализация `IUnitOfWork` инкапсулирует соединение с базой данных для контроля его жизненного цикла.
- **File System:** Реализация `IFileStorage` через классы `System.IO`.
- **Перехват исключений:** Любые `SqlException`, `IOException` и другие системные ошибки отлавливаются здесь, **логируются через Serilog** (с уровнем Error/Fatal и полным StackTrace), а затем безопасно конвертируются в `Result.Failure(InfrastructureErrors...)`.

5. Проектирование базы данных SQLite

Схема спроектирована для эффективной работы с реляционными связями:

- `Photos`: Id, RealPath (UNIQUE), FileHash, Width, Height, AddedAt.
- `Folders`: Id, Name, ParentFolderId (FK -> Folders.Id).
- `Albums`: Id, Name, FolderId (FK -> Folders.Id).
- `Tags`: Id, Name (UNIQUE).

- `AlbumPhotos`: AlbumId, PhotoId (Composite PK, ON DELETE CASCADE).
- `PhotoTags`: PhotoId, TagId (Composite PK, ON DELETE CASCADE).

Обязательное требование: При каждом открытии соединения в `Infrastructure` необходимо выполнять `PRAGMA foreign_keys = ON;` для активации ссылочной целостности SQLite. Этот вызов контролируется внутри реализации `IUnitOfWork`.

6. Нефункциональные требования к коду и тестированию

- **Стиль кода:** Строгое соблюдение официального Style Guide от Microsoft (C#).
- **Архитектурные тесты (ArchUnitNET):** Наличие тестов, проверяющих направления зависимостей (`Domain` не знает про `Infrastructure` и `Serilog`).
- **Unit-тесты (xUnit):** Покрытие доменных сущностей и `UseCase` классов.
- **Стратегия логирования (Serilog):** Структурированное логирование в консоль и/или файл. Сообщения должны содержать контекст (например, `PhotoId`, `UseCaseName`). Ошибки логируются с сохранением оригинальных кодов из справочников `Errors`.
- **Обработка бизнес-ошибок (ROP):** Обработка ошибок производится с помощью паттерна Result. Использование реестров ошибок для каждого слоя:
  - **`DomainErrors`:** бизнес-правила (например, `DomainErrors.Tag.TooLong`).
  - **`ApplicationErrors`:** ошибки потока (например, `ApplicationErrors.Photo.FileNotFoundOnImport`).
  - **`InfrastructureErrors`:** системная грязь (например, `InfrastructureErrors.FileStorage.DiskFull`). Оборачивается в `Result` после записи в лог.
- **Защита инвариантов:** Доменная модель не позволяет перевести систему в невалидное состояние. Валидация происходит строго внутри `Domain`.

7. Этапы разработки

1. **Domain & Testing:** Создание сущностей (`Photo`, `Album`, `Folder`) с инкапсуляцией. Реализация возвратов паттерна `Result` / `ResultVoid` и написание Unit-тестов.
2. **Архитектурные тесты:** Настройка ArchUnitNET для фиксации слоев.
3. **Скелет БД:** Создание DDL-скриптов для генерации файла SQLite со всеми таблицами.
4. **Data Access & Infrastructure:** Конфигурация **Serilog** в DI-контейнере. Реализация чистой версии `UnitOfWork`, CRUD-операций и перехвата `Exception` с записью в лог.
5. **Use Cases:** Реализация прикладного слоя (`Application`), внедрение логгера (`ILogger<T>`) в Use Cases и оркестрация вызовов.

6. **Интеграционное тестирование:** Запуск In-Memory SQLite (или тестового файла БД), проверка полного цикла работы приложения и анализ сгенерированных файлов логов на предмет корректного отслеживания ошибок.

Руководство по использованию паттерна Result и его Fluent Api

1. Базовые типы: Контейнеры результатов

Паттерн строится на двух неизменяемых структурах данных. Они фиксируют состояние операции в момент ее завершения.

1.1. ResultVoid (Операция без возвращаемых данных)

Используется для методов, которые выполняют действие, изменяющее состояние системы, но не генерируют новых данных для возврата (аналог `void`).

Свойства:

- `IsSuccess` (`bool`): Возвращает `true`, если операция завершена без ошибок.
- `IsFailure` (`bool`): Возвращает `true`, если произошел сбой. Всегда равно `!IsSuccess`.
- `Error` (`Error`): Объект с информацией об ошибке (код и сообщение). При успешном выполнении содержит `Error.None`.

Сценарии использования:

- Сохранение записи в базу данных.
- Удаление файла (например, удаление фотографии с диска).
- Отправка сетевого запроса или email-уведомления.

1.2. Result (Операция с возвращаемыми данными)

Используется, когда метод должен вычислить, найти или создать объект типа `T` и передать его вызывающему коду.

Свойства:

- Включает все свойства `ResultVoid` (`IsSuccess`, `IsFailure`, `Error`).
- `Value` (`T`): Итоговые данные. **Критическое правило:** обращение к `Value` допускается только при `IsSuccess == true`. В противном случае значение равно `default` или `null`.

Сценарии использования:

- Поиск сущности в репозитории (например, поиск `Photo` по идентификатору).

- Создание нового объекта на основе входных параметров.
- Парсинг строки в числовой формат.

---

2. Инициализация и создание объектов

Чтобы начать работу, данные необходимо поместить в контекст `Result`.

Вариант А: Прямое использование статических методов

Обеспечивает максимальную очевидность намерений в коде.

```
// Для операций без возвращаемого значения
ResultVoid deleteResult = ResultVoid.Success();
ResultVoid deleteError = ResultVoid.Failure(new Error("File.Locked", "Файл занят
другим процессом"));

// Для операций с данными
Result<int> calcResult = Result<int>.Success(256);
Result<int> calcError = Result<int>.Failure(new Error("Math.DivisionByZero",
"Деление на ноль"));
```

Вариант Б: Неявное приведение типов (Implicit Conversion)

Язык C# позволяет возвращать непосредственно значение или объект ошибки. Компилятор автоматически трансформирует их в `Result<T>`.

```
public Result<Photo> GetPhoto(Guid id)
{
    Photo photo = _repository.Find(id);

    if (photo == null)
        return new Error("Photo.NotFound", "Фотография не найдена"); //
Автоматически конвертируется в Result<Photo>.Failure

    return photo; // Автоматически конвертируется в Result<Photo>.Success
}
```

---

3. Методы расширения: Построение цепочек вызовов (Pipeline)

Методы расширения позволяют организовать линейный поток выполнения кода.

Главный принцип механики: Каждый метод расширения перед выполнением своей логики проверяет состояние входящего `Result`. Если текущее состояние — `IsFailure` (ошибка), метод не выполняет переданную в него функцию, а просто возвращает текущую ошибку дальше по цепочке. Это устраняет необходимость писать вложенные проверки `if`.

3.1. Метод `ToResult`

Назначение: Точка входа в цепочку. Оборачивает потенциально пустое значение (`null`) в защищенный контейнер.

- **Параметр `value`:** Переменная или результат функции, которые нужно проверить. «Параметр value не нужно писать вручную внутри скобок. Им становится тот объект, у которого вы вызываете метод через точку. Если этот объект окажется null, метод ToResult увидит это и создаст Result со статусом "Ошибка". Если объект существует — он будет бережно упакован внутрь контейнера».
- **Параметр `errorIfNull`:** Объект ошибки, который будет сгенерирован, если `value` равно `null`.

Сценарий: Загрузка данных из внешнего источника, где гарантия существования объекта отсутствует.

```
// Ищем альбом. Если возвращается null, создается ошибка Album.NotFound.
Result<Album> albumResult = _context.Albums.FirstOrDefault(a => a.Id == id)
    .ToResult(new Error("Album.NotFound", "Альбом не существует"));
```

3.2. Метод `Ensure`

Назначение: Строгая проверка условий для текущих данных.

- **Сигнатура:** `Ensure(Func<TValue, bool> predicate, Error error)`
- **Параметр `predicate`:** Делегат (функция), принимающий текущее значение и возвращающий `true` или `false`.
- **Параметр `error`:** Ошибка, которая прервет выполнение, если предикат вернет `false`.

Сценарий: Проверка корректности входных данных (валидация) до выполнения ресурсоемких операций.

```
public Result<EvaluationList> CreateList(string title, int maxItems)
{
    return title.ToResult(Errors.TitleNull)
        .Ensure(t => t.Length >= 3, new Error("Validation.TitleTooShort", "Длина должна быть больше 3 символов"))
        .Ensure(_ => maxItems > 0, new Error("Validation.InvalidLimit", "Лимит элементов должен быть положительным"))
        .Transform(validTitle => new EvaluationList(validTitle, maxItems));
}
```

3.3. Метод `Transform`

Назначение: Изменение структуры или типа данных при условии успешного выполнения предыдущих шагов. Внутренняя функция преобразования не может прервать цепочку ошибкой.

- **Сигнатура:** `Transform<TValue, TNextValue>(Func<TValue, TNextValue> mapper)`
- **Параметр `mapper`:** Делегат, который принимает текущие данные (`TValue`) и возвращает новые данные (`TNextValue`).

Сценарий: Конвертация тяжелых сущностей базы данных в легковесные DTO-объекты для передачи по сети.

```
Result<PhotoDto> dtoResult = _repository.GetPhoto(id)
    .Transform(photo => new PhotoDto
    {
        Id = photo.Id,
        FilePath = photo.Path
    });
```

3.4. Метод `Then`

Назначение: Последовательное выполнение операций, где каждая последующая операция зависит от успешности предыдущей и сама способна вернуть ошибку.

- **Сигнатура:** `Then<TValue, TNextValue>(Func<TValue, Result<TNextValue>> nextStep)` (также имеет перегрузки для работы с `ResultVoid`).
- **Параметр `nextStep`:** Функция, которая принимает текущее значение и возвращает новый объект `Result` или `ResultVoid`.

Сценарий: Координация нескольких доменных сервисов.

```
public ResultVoid AssignPhotoToAlbum(Guid photoId, Guid albumId, Guid currentUserId)
{
    // 1. Получаем фото
    return _photoRepository.Get(photoId).ToResult(Errors.PhotoNotFound)
        // 2. Выполняем проверку прав (возвращает ResultVoid)
        .Then(photo => _securityService.CheckAccess(photo, currentUserId))
        // 3. Если права есть, получаем альбом (возвращает Result<Album>)
        .Then(() => _albumRepository.Get(albumId).ToResult(Errors.AlbumNotFound))
        // 4. Если альбом найден, связываем их и сохраняем (возвращает ResultVoid)
        .Then(album => _albumRepository.AddPhoto(album.Id, photoId));
}
```

3.5. Методы `OnSuccess` и `OnFailure`

Назначение: Выполнение операций, не влияющих на основную логику обработки данных и состояние контейнера.

- **Сигнатуры:** `OnSuccess(Action<TValue> action)` и `OnFailure(Action<Error> action)`.
- **Параметр `action`:** Делегат, выполняющий конечное действие.

Сценарий: Логирование, аудит, обновление счетчиков в памяти.

```
var result = processData()
    .OnSuccess(data => Console.WriteLine($"Обработано байт: {data.Length}"))
    .OnFailure(error => Console.WriteLine($"Операция отклонена.
Причина: {error.Code}"));
```

3.6. Метод `Finally`

Назначение: Терминальная (конечная) операция цепочки. Извлекает данные из контекста `Result` и обязывает программиста предоставить алгоритм обработки как для успешного исхода, так и для ошибки. Возвращает итоговое значение единого типа.

- **Сигнатура:** `Finally<TValue, TLanding>(Func<TValue, TLanding> success, Func<Error, TLanding> failure)`
- **Параметр `success`:** Делегат, описывающий, во что превратить данные типа `TValue`.
- **Параметр `failure`:** Делегат, описывающий, во что превратить объект `Error`.

Сценарий: Преобразование результатов доменного слоя в HTTP-ответы контроллера.

```
public IActionResult GetEvaluationList(Guid id)
{
    return _listService.GetById(id)
        .Finally(
            success: list => Ok(list),
            failure: error => error.Code == "List.NotFound" ? NotFound(error)
: BadRequest(error)
        );
}
```

---

4. Комплексные сценарии применения

Данный пример демонстрирует полный цикл обработки бизнес-задачи с применением рассмотренных методов. Сценарий: удаление фотографии пользователя.

Требования к операции:

1. Файл должен существовать в БД.
2. Текущий пользователь должен являться владельцем файла.
3. Физический файл должен быть удален с диска (операция может завершиться сбоем).
4. Запись в БД должна быть удалена (операция может завершиться сбоем).
5. Необходима запись в лог.

Реализация:

```
public ResultVoid DeleteUserPhoto(Guid photoId, User currentUser)
{
    return _repository.FindById(photoId)
        .ToResult(new Error("Photo.NotFound", "Файл не существует в системе"))

        .Ensure(
            predicate: photo => photo.OwnerId == currentUser.Id,
            error: new Error("Access.Denied", "Вы не являетесь владельцем файла")
        )

        .Then(photo => _fileSystem.DeletePhysicalFile(photo.Path)) //
```

```
Возвращает ResultVoid

        .Then(() => _repository.RemoveFromDatabase(photoId)) // Выполняется, только
если файл удален с диска

        .OnSuccess(() => _logger.LogInformation("Процесс удаления
успешно завершен"))

        .OnFailure(error => _logger.LogError($"Сбой удаления: Код {error.Code},
Сообщение: {error.Message}"));
}
```

Анализ механики: Метод полностью лишен операторов ветвления. Если на шаге `Ensure` пользователь не является владельцем, объект `Result` получает статус `IsFailure` с ошибкой "Access.Denied". Методы `Then` (удаление файла и записи БД) обнаруживают этот статус и пропускают свое выполнение. Поток управления мгновенно переходит к методу `OnFailure`.

Шпаргалка по выбору методов `Result`

Метод	Главный вопрос: "Что мне нужно сделать?"	Что происходит при Успехе (на входе)?	Что происходит при Ошибке (на входе)?	Пример сценария
`ToResult`	Как мне начать цепочку, если объект пришел из базы и может быть `null`?	Создает `Success` и кладет данные внутрь.	Создает `Failure` с указанной вами ошибкой.	Поиск пользователя: `_db.Find(id).ToResult(`
`Ensure`	Как проверить правило (длину, права доступа, лимиты)?	Запускает проверку. Если `true` — пропускает дальше. Если `false` — выдает новую ошибку.	**Игнорируется.** Ошибка просто летит дальше по цепочке.	Проверка возраста: `.E` `u.Age >= 18, Errors.To`

Метод	Главный вопрос: "Что мне нужно сделать?"	Что происходит при Успехе (на входе)?	Что происходит при Ошибке (на входе)?	Пример сценария
`Transform`	Как превратить сущность в DTO (и я уверен, что тут не будет сбоев)?	Выполняет трансформацию и кладет новые данные в новый `Result`.	**Игнорируется.** Старая ошибка летит дальше по цепочке.	Создание DTO: `.Transf` `UserDto(u.Name))`
`Then`	Как вызвать следующий метод, который тоже может выдать ошибку?	Выполняет следующий метод и берет его `Result` за основу для дальнейших шагов.	**Игнорируется.** Старая ошибка летит дальше по цепочке.	Списание денег: `.Then` `_bank.Charge(u.Id, 100`
`OnSuccess` / `OnFailure`	Как записать лог или почистить кэш, не ломая основную логику?	`OnSuccess` — выполняется. `OnFailure` — игнорируется. Данные не меняются.	`OnFailure` — выполняется. `OnSuccess` — игнорируется. Данные не меняются.	Логирование: `.OnSucce` `_logger.Info("Успех"))`
`Finally`	Как выйти из конвейера и вернуть финальный HTTP-ответ пользователю?	Выполняет ветку `success` и возвращает итоговый объект.	Выполняет ветку `failure` и возвращает итоговый объект.	Ответ контроллера: `.` `Ok(), err => BadReques`

Быстрый алгоритм принятия решений:

1. Вы только достали данные? → **`ToResult`**
2. Шаг может провалиться из-за неверных данных? → **`Ensure`**
3. Шаг может провалиться, потому что вызывает сторонний сервис/БД? → **`Then`**

4. Шаг 100% безопасен и просто меняет форму данных? → **`Transform`**
5. Шаг вообще ничего не меняет, а просто "смотрит"? → **`OnSuccess`** / **`OnFailure`**
6. Пора отдавать данные наружу из вашего слоя Application? → **`Finally`**

5. Глоссарий терминов

- **Паттерн Result:** Архитектурный подход инкапсуляции результатов выполнения кода, заменяющий обработку исключений строгой типизацией успешных данных и состояний ошибок.
- **Линейный поток выполнения (Pipeline):** Организация кода в виде последовательности вызовов, где выходные данные одной функции служат входными данными для следующей, формируя единую цепь операций.
- **Делегат (Func / Action):** Ссылка на метод с определенной сигнатурой, передаваемая в качестве параметра. Позволяет отложить выполнение кода.
- **Побочный эффект (Side Effect):** Операция, взаимодействующая с внешним состоянием (сеть, консоль, диск), но не модифицирующая данные внутри текущего потока вычислений (методы `OnSuccess` / `OnFailure`).
- **Терминальный метод:** Функция, завершающая линейный поток вычислений и извлекающая данные из внутренних структур (контейнеров) во внешний код.
- **Неявное приведение (Implicit Conversion):** Механизм языка C#, позволяющий компилятору автоматически преобразовывать один тип данных в другой (например, сущность БД в объект `Result<T>`) без явного указания приведения в коде.